



VISIONGUARD AI

Technical Implementation Report

Local-first camera-based vision screening, viewing-distance guidance, and digital eye-fatigue monitoring

DOCUMENT	DETAIL
Version	1.0.0 Public Preview
Date	20 July 2026
Course project	UCUG1907 - Design Thinking - Team Assignment 5
Live prototype	projectdesignthinking.pages.dev

Team: Aslbek MAKHMUDOV | Kassymkhan JUVAROV | Qianrui CUI | Sanzhar ORAZBAY | Ziteng WANG

01 Executive Summary

VisionGuard AI is a cross-platform screening and guidance prototype that combines local facial-landmark inference, screen and camera calibration, a Landolt C visual-acuity workflow, blink monitoring, viewing-distance monitoring, and explainable rule-based guidance. Its primary design objective is to make basic eye-health self-screening accessible without dedicated hardware while preserving a strong local-processing privacy boundary.

The system is implemented as a shared React and TypeScript application. The same production web bundle is deployed to Cloudflare Pages, embedded in an Android application through Capacitor, and packaged for Windows, Linux, and macOS through Electron. Camera frames are processed in the client; the application does not require an account or application server for its core workflow.

Important interpretation: the prototype produces screening indicators and behavioral guidance, not a diagnosis. Software correctness has been tested, but sensor accuracy and clinical validity still require controlled participant studies using calibrated reference equipment.

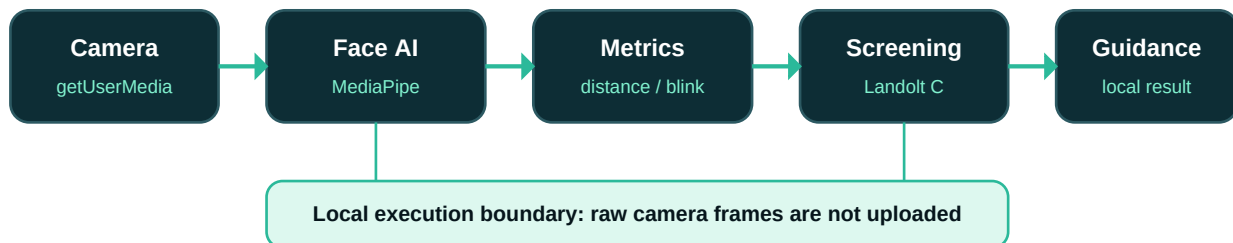
Key technical characteristics

- Local MediaPipe Face Landmarker inference with GPU-first and CPU fallback execution.
- Individual camera-distance calibration based on observed inter-eye scale at a known reference distance.
- Physical optotype sizing derived from screen pixels per millimeter and the five-arcminute acuity relationship.
- Adaptive blink detection using a participant-specific eye-aspect-ratio baseline.
- Rule-based, explainable fatigue and risk synthesis with explicit confidence limitations.
- Optional minimal local persistence; raw images, raw video, and per-question answers are not retained.
- Camera-free manual and demonstration routes for accessibility and classroom presentation.

Layer	Primary technology	Responsibility
Presentation	React 18, TypeScript, CSS	Six-stage onboarding, calibration, test, monitoring, results, localization
Vision	MediaPipe Tasks Vision 0.10.35	Single-face landmark detection in video mode
Web build	Vite 5	Static production bundle with relative asset paths
Desktop	Electron 43, electron-builder	Secure local protocol, OS camera permissions, installers and archives
Android	Capacitor 8, Gradle	Secure WebView host, filesystem export, native share sheet
Persistence	Web Storage / local files	Optional structured summaries and user preferences

02 System Architecture

The application follows a client-only pipeline. Each camera frame is decoded by the browser or native web runtime, processed by the bundled face landmark model, converted into geometric measurements, and then reduced to user-facing metrics. Only derived values flow into screening and guidance logic.



Runtime flow

- The user reviews consent and explicitly enables camera access.
- The runtime requests a user-facing camera at an ideal 640 x 480 resolution and up to 30 frames per second.
- MediaPipe detects one face and returns normalized landmarks for the current frame.
- The geometry layer derives inter-eye span, eye aspect ratio, head alignment, face centering, and frame aspect ratio.
- Calibration and temporal filters convert frame-level observations into distance, blink, and tracking metrics.
- The screening layer controls the Landolt C test and converts answers into Snellen, decimal-acuity, and logMAR outputs.
- The guidance layer combines symptoms, screening outputs, and monitoring metrics into explainable next steps.

Shared-code deployment

The web, Android, and desktop versions use the same compiled `dist` directory. Platform wrappers are intentionally thin: Capacitor exposes Android filesystem/share capabilities, while Electron supplies a secure application origin, camera permission handling, navigation restrictions, tray behavior, and packaging. This architecture keeps feature behavior synchronized across platforms.

Deployment	Host/runtime	Camera security context	Primary artifact
Web	HTTPS static hosting	Browser secure context	Cloudflare Pages deployment
Android	Capacitor WebView	Custom HTTPS scheme	APK / Android bundle
Windows	Electron / Chromium	Trusted local app protocol	Installer / portable package
Linux	Electron / Chromium	Trusted local app protocol	ApplImage, DEB, or archive
macOS	Electron / Chromium	OS camera entitlement	Intel / Apple Silicon application

03 Local Vision Engine

Model loading and inference

The application loads a bundled MediaPipe Face Landmarker task model and local WebAssembly runtime. A remote official model/runtime location is retained only as a fallback. Inference runs in video mode, tracks a maximum of one face, and uses confidence thresholds of 0.55 for face detection and presence and 0.50 for tracking. GPU delegation is attempted first; model creation automatically retries with CPU delegation if required.

Geometric measurements

Measurement	Implementation	Purpose
Inter-eye span	Euclidean distance between outer eye-corner landmarks	Relative camera distance
Eye aspect ratio (EAR)	Two vertical eyelid distances divided by twice the horizontal eye width	Blink state
Centering	Eye midpoint constrained to the central image region	Reject unreliable pose
Roll/level	Vertical eye difference divided by inter-eye span	Reject tilted face
Yaw symmetry	Ratio of nose-to-left-eye and nose-to-right-eye distances	Reject strong side-facing pose
Frame aspect	Camera frame width divided by height	Invalidate calibration after rotation/crop change

Lighting analysis

Once per second, the video is downsampled to 32 x 24 pixels. Mean luminance is computed using the weighted RGB expression $0.2126R + 0.7152G + 0.0722B$, normalized to 0-1. Values between 0.18 and 0.92 are accepted. This lightweight test detects broadly underexposed or overexposed frames; it is not a photometric measurement.

Temporal behavior

Landmark analysis is scheduled at approximately 110 ms intervals. Distance estimates use a rolling median of up to seven valid samples. Tracking time advances only while valid face data is available, with each frame interval capped at 500 ms to prevent a long browser stall from being counted as continuous observation.

The engine reports explicit states such as idle, requesting camera, loading model, running, paused, demonstration, and error. Camera tracks are stopped when monitoring ends; permission, missing-device, camera-in-use, model-loading, and generic engine errors are mapped to recoverable user messages.

04 Distance and Screen Calibration

Camera-distance calibration

A monocular webcam cannot provide an absolute physical distance from facial size alone without a reference. VisionGuard therefore asks the user to remain at a known distance (60 cm by default) while it collects valid, well-lit, forward-facing samples for approximately 2.2 seconds. At least eight samples are required. Median inter-eye scale and frame aspect ratio become the session calibration baseline.

```
Distance estimate
D_current = (S_reference x D_reference) / S_current
where S is normalized inter-eye width and D is distance in centimeters.
```

The inverse-scale relationship is a practical approximation under a stable camera field of view. It is invalidated when device orientation changes or the frame aspect ratio differs from the calibrated value by more than approximately 2.5 percent. Camera calibration is intentionally session-bound because zoom, crop, resolution, and orientation may change between sessions.

Screen calibration

The visual-acuity test requires the physical size of each optotype. VisionGuard supports exact physical matching against an ID-1-size card (85.60 x 53.98 mm) or TD-3 passport page (125 x 88 mm), plus a convenient screen-diagonal estimate for situations where no reference object is available.

```
Diagonal estimate
pixel_diagonal = sqrt(width_css^2 + height_css^2)
physical_diagonal_mm = diagonal_inches x 25.4
pixels_per_mm = pixel_diagonal / physical_diagonal_mm
```

Method	Strength	Primary limitation
Physical card	Direct on-screen physical match	Depends on accurate manual alignment and browser zoom
Passport TD-3	Larger reference, potentially easier to align	Passport availability and careful handling
Screen diagonal	No physical object required	Approximate: depends on correct model size and reported CSS viewport

For the diagonal method, users should enter the display diagonal, not the device width or height. Rounded-corner displays may have a slightly smaller viewable area than the marketed rectangular diagonal.

05 Visual-Acuity Screening

The acuity workflow uses a four-orientation Landolt C optotype. The gap direction is randomized and never repeated immediately. Users respond by touch controls, arrow keys, or WASD keys. Binocular, left-eye, and right-eye modes are supported; covering one eye remains the user's responsibility.

Physical optotype sizing

A standard 20/20 optotype subtends five arcminutes at the eye. Larger Snellen denominators scale this angular size. The software converts the resulting millimeter dimension into pixels using the screen calibration.

```
theta = 5 arcmin x (denominator / 20)
size_mm = 2 x distance_mm x tan(theta / 2)
size_px = size_mm x pixels_per_mm
```

Adaptive test procedure

Stage	Behavior
Readiness	Requires symptom step, screen calibration, and either valid live distance/pose/light or manual confirmation
Levels	20/200, 20/125, 20/80, 20/63, 20/50, 20/40, 20/32, 20/25, and 20/20
Trials	Three randomized directions per level
Pass rule	At least two correct answers out of three
Termination	Stops at the first failed level or after passing 20/20
Outputs	Snellen value, decimal acuity, logMAR, response accuracy, average observed distance, and calibration flags

During live-camera testing, interaction pauses whenever distance, pose, face presence, or lighting leaves the accepted range. In manual mode, distance measurements are recorded as unavailable rather than simulated. Demonstration results are explicitly marked and excluded from personalized health guidance.

Result conversions

For a Snellen denominator n , decimal acuity is calculated as $20 / n$, while logMAR is calculated as $\log_{10}(n / 20)$. These are mathematical representations of the prototype result and do not establish clinical equivalence with a professionally administered examination.

06 Blink, Fatigue, and Risk Logic

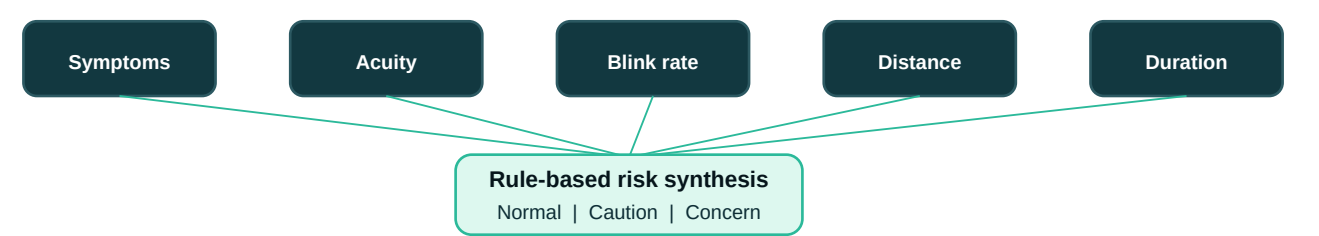
Adaptive blink detection

Each eye's aspect ratio is calculated from six landmarks, and the two values are averaged. The system maintains up to 160 open-eye samples and uses the 80th percentile as an individual baseline. The blink threshold is the larger of 0.12 and 62 percent of that baseline. A closure becomes a blink only when it lasts between 65 and 900 ms.

Blink rate is reported only after at least 20 seconds of tracked data. This adaptive strategy is more robust than one global threshold, but glasses, eyelid anatomy, head angle, image blur, low light, and low frame rate can still produce missed or false events.

Fatigue score

The fatigue score is a transparent heuristic on a 0-100 scale. It is the clipped sum of a low-blink penalty (maximum 45 points), a too-close-duration penalty (maximum 35 points), and a long-session penalty (maximum 20 points). Scores below 30 are labelled low, 30-59 moderate, and 60 or above high after at least 30 tracked seconds.



Explainable screening risk

Risk synthesis combines selected symptoms, acuity, blink rate, distance behavior, and fatigue. Red-flag symptoms (eye pain, double vision, or flashes/floaters) produce an urgent-care recommendation. Acuity worse than 20/40, high fatigue, or multiple non-red signals produce a concern classification. One non-red signal produces caution. Every assessment also reports confidence limitations, including missing calibration, insufficient tracking, demonstration mode, absent distance data, or legacy records without metric snapshots.

Operational threshold	Value
Too close for continuous monitoring	Below 45 cm
Supported result-distance range	40-80 cm
Low blink rate indicator	Below 10 blinks/min after at least 30 tracked seconds
Session too-close indicator	More than 25% of tracked time
Acuity concern	Worse than 20/40 or unable to pass the first level
Continuous report window	20 minutes

07 Privacy, Security, and Data Handling

VisionGuard uses a local-first data model. Raw video frames are consumed transiently by the browser rendering and inference pipeline. The application does not contain an upload endpoint for images or video, and its core screening workflow does not require an account, cloud database, or application server.

Data categories

Data	Default treatment	Persistence
Raw camera video / images	Processed locally and discarded	Never intentionally stored
Facial landmarks	Used in memory for current-frame geometry	Not persisted
Symptoms	Used for the current explanation	Not retained in saved result history
Per-question acuity answers	Used to compute the final result	Removed from persisted summaries
Derived metrics	Distance, blink, fatigue, calibration flags	Optional minimal local summaries
Preferences	Language, consent, reminder, calibration	Local storage on the device

Desktop hardening

- A privileged custom visionguard://app scheme provides a secure local origin.
- Renderer sandboxing and context isolation are enabled; Node.js integration and webviews are disabled.
- A restrictive Content Security Policy limits scripts, media, network connections, frames, forms, and objects.
- Only video-camera permission from the trusted main page is accepted; microphone and display capture are denied.
- Unexpected navigation is blocked and approved external HTTP(S), email, or telephone links open outside the app.
- Resolved resource paths are checked to remain inside the packaged distribution directory.

User control

Camera processing begins only after consent and a user action. Users can pause, stop, or exit monitoring. Disabling optional result storage immediately deletes saved summaries and reminders. Export uses a JSON download on the web and a temporary cache file or native share sheet on Android. The application continues in memory when storage is blocked.

Local-first does not mean zero risk: the operating system, browser, device administrator, backups, malware, or another person with device access may still expose locally stored summaries. Production deployments should document retention, device management, consent language, and applicable institutional privacy requirements.

08 Delivery, Verification, and Current Evidence

Build and test status

The current source tree was revalidated on 20 July 2026. TypeScript compilation completed without errors, all 128 unit tests across 16 test files passed, and the Vite production build completed successfully with 155 transformed modules. The generated main JavaScript bundle was approximately 498.91 kB (162.59 kB gzip), and the main CSS bundle was approximately 82.15 kB (16.12 kB gzip).

Verification activity	Current result	What it demonstrates
TypeScript typecheck	Passed	Static type consistency
Unit tests	128/128 passed	Deterministic calibration, acuity, fatigue, risk, storage, locale, and runtime logic
Production build	Passed	Compilable deployable web assets
Earlier browser E2E	Passed	Manual 27-response flow, privacy deletion, localization, QR generation, responsive layouts
MediaPipe smoke test	Passed	Local task model and WASM runtime load in Chromium

Evidence not yet established

Automated testing proves software behavior, not clinical validity or measurement accuracy. The following project targets remain pending external validation:

Target	Required study
Acuity within one chart line of a reference test	Representative participant study against a standardized professionally administered chart
Blink-event F1 greater than 0.85	Frame/event annotations across participants, glasses, poses, cameras, and lighting
Distance error within 5 cm from 40-80 cm	Calibrated physical measurements across devices, zoom/crop states, postures, and orientations

Packaging status

The public preview is distributed as a web application and platform packages. Android currently uses a test/debug build; macOS packages are unsigned and not notarized; desktop and Android packages require representative real-device acceptance tests. Signing, notarization, store compliance, accessibility review, and update delivery are release engineering tasks rather than core algorithm requirements.

09 Limitations and Recommended Next Steps

Known technical limitations

- Distance estimation is relative and monocular; it depends on calibration quality and a stable camera field of view.
- Screen-diagonal calibration is convenient but approximate and can be affected by browser zoom, CSS viewport reporting, operating-system scaling, and rounded display corners.
- Blink detection may degrade with glasses, reflections, low light, eyelid variation, motion blur, occlusion, and head rotation.
- The acuity test cannot enforce monocular occlusion, room illumination, participant honesty, or a clinically standardized testing environment.
- Fatigue and risk scores are intentionally heuristic and should not be interpreted as disease probabilities.
- The browser foreground model limits monitoring when a mobile operating system suspends or throttles the application.
- A camera-free manual path improves access but cannot supply objective distance, blink, pose, or lighting measurements.

Recommended validation roadmap

Priority	Activity	Acceptance evidence
1	Distance bench study	Mean absolute error and 95% limits across 40, 50, 60, 70, and 80 cm
2	Blink annotation study	Precision, recall, F1, and subgroup results for glasses and lighting
3	Acuity comparison study	Agreement by chart line and failure analysis by display/calibration method
4	Think-aloud usability test	Task completion, calibration comprehension, recovery behavior, and interpretation accuracy
5	Platform acceptance matrix	Camera permission, orientation, resume, export, installer, and accessibility results
6	Release hardening	Signed artifacts, macOS notarization, reproducible builds, checksums, and update policy

Conclusion

VisionGuard AI demonstrates the technical feasibility of a local-first, cross-platform eye-health screening and screen-use guidance experience using commodity cameras and displays. Its strongest engineering properties are a shared multi-platform codebase, explicit calibration, explainable calculations, user-controlled camera access, minimal local persistence, and clear fallback paths. The next stage should prioritize calibrated accuracy studies and real-user comprehension rather than adding more diagnostic claims or opaque scoring.

Final safety statement: VisionGuard AI is an educational screening prototype. It does not diagnose eye disease, prescribe treatment, or replace an examination by an optometrist or ophthalmologist.

A Implementation Reference

Core source modules

Module	Role
src/hooks/useVisionEngine.ts	Camera lifecycle, model loading, frame analysis, temporal metrics, calibration
src/lib/vision.ts	EAR, inter-eye span, pose checks, frame compatibility, distance equation
src/lib/screenCalibration.ts	Physical-reference and diagonal screen calibration
src/lib/acuity.ts	Physical optotype sizing and result conversions
src/components/VisionTest.tsx	Landolt C state machine, readiness gates, input, pass/fail progression
src/lib/fatigue.ts	Blink rate, fatigue score, bands, behavioral recommendations
src/lib/risk.ts	Multi-signal risk classification, confidence, limitations, next steps
src/lib/storage.ts	Local persistence and web/native JSON export
electron/main.mjs	Desktop protocol, permissions, CSP, navigation, tray, lifecycle
capacitor.config.ts	Android secure WebView and application configuration

External technical references

- [Google AI Edge - Face Landmarker for Web](https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker/web_js)
https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker/web_js
- [W3C - Media Capture and Streams](https://www.w3.org/TR/mediacapture-streams/)
<https://www.w3.org/TR/mediacapture-streams/>
- [Electron - Security Recommendations](https://www.electronjs.org/docs/latest/tutorial/security)
<https://www.electronjs.org/docs/latest/tutorial/security>
- [Capacitor Documentation](https://capacitorjs.com/docs)
<https://capacitorjs.com/docs>
- [Cloudflare Pages Documentation](https://developers.cloudflare.com/pages/)
<https://developers.cloudflare.com/pages/>
- [Project repository](https://github.com/Inv1dx/ProjectDesignThinking)
<https://github.com/Inv1dx/ProjectDesignThinking>
- [Live prototype](https://projectdesignthinking.pages.dev/)
<https://projectdesignthinking.pages.dev/>

This report describes the implementation present in the reviewed Version 1.0.0 project source and the validation executed on 20 July 2026. Future releases may change algorithms, thresholds, packaging, and validation status.